

Cowboy: A Layer-1 Blockchain for Autonomous Agents

Technical Whitepaper

Charles DePue, Patrick Mandia, Tony Xia, Martin Aceto

April 2026 — Draft v0.9.2

1 Abstract

Cowboy is a general-purpose Layer-1 blockchain that combines a **Python-based actor-model execution environment** with **proof-of-stake consensus**, a **market for verifiable off-chain computation**, and **protocol-native persistent off-chain storage**. Smart contracts on Cowboy are **actors**: Python programs with private state, a mailbox for messages, and chain-native timers for autonomous scheduling. For heavy tasks like LLM inference or web requests, Cowboy integrates a decentralized network of **Runners** who execute jobs and attest to results under selectable trust models: N-of-M consensus, TEEs, and (in V2) ZK-proofs. For large artifacts and durable runner state, Cowboy provides **Runner Attached Storage (RAS)**: account-scoped volumes stored across CBFS Relay Nodes and anchored on-chain through compact storage commitments.

Cowboy introduces a **dual-metered gas model**, separating pricing for computation (**Cycles**) and data (**Cells**) into independent, EIP-1559-style fee markets. Security is provided by **Simplex BFT** consensus with **proof-of-stake**, fast finality, and mandatory proposer rotation.

This document specifies Cowboy’s complete technical architecture, state transition function, economic mechanisms, consensus protocol, and all implementation parameters.

2 Introduction

Cowboy is designed to enable autonomous agents by providing a native blockchain execution environment optimized for asynchronous, Python-based applications. This document provides complete technical specifications for implementers, auditors, and protocol developers.

Ethereum gave us programmable money, but its execution model is fundamentally reactive — smart contracts sit inert until an externally-controlled system calls them. Cowboy introduces a new unit of execution: the **actor**, an immortal, stateful program that schedules itself into the future, bids for its own blockspace, and runs indefinitely without a human in the loop. No amount of keeper networks or cron-job infrastructure on Ethereum provides a first-class primitive for autonomous, self-funding, self-scheduling programs. This new model requires a new protocol.

Cowboy further innovates by adopting Python as its execution language, replacing the domain-specific languages required by existing chains (Solidity on Ethereum, Rust on Solana). Python’s ubiquity in AI and general-purpose development reduces the barrier to entry for both human developers and AI coding agents, which already generate Python more reliably than niche smart contract languages.

For architectural rationale and design decisions, see the [Design Decisions Overview](#).

3 Architectural Overview

This section is descriptive and non-binding. Normative requirements are in §§1–17.

3.1 Terminology

- **Actor:** A Python program with persistent key/value state and a mailbox.
- **Message:** A datagram delivered to an actor handler.
- **Cycle:** Unit of metered on-chain compute.
- **Cell:** Unit of metered bytes (1 cell = 1 byte).
- **Runner:** Off-chain worker that executes a job and returns an attested result.
- **Entitlement:** A permission governing an actor’s or runner’s capabilities.
- **GBA:** Gas Bidding Agent — an actor that dynamically bids for timer execution on behalf of another actor.

3.2 Key Features

Cowboy implements four core technical features:

- **Deterministic Python Actors.** Smart contracts on Cowboy are actors: Python programs with private state, a mailbox for asynchronous messages, and chain-native timers. Actors execute in a sandboxed Python VM (PVM) pinned to Python 3.11.8 with deterministic floating-point (softfloat), disabled GC, fixed hash seed, and a whitelisted module set. Reentrancy is depth-capped to 32; per-call memory is bounded at 10 MiB.
- **Native Timers & Scheduler.** Actors schedule their own future execution via `set_timer` and `set_interval` without external keeper infrastructure. The scheduler uses a tiered calendar queue and a Gas Bidding Agent (GBA) mechanism that dynamically bids for timer execution at block time. Anti-DoS measures include progressive deposits, exponential same-block surcharges, per-actor timer caps, and a dynamic timer basefee.
- **Verifiable Off-Chain Compute.** A decentralized Runner marketplace executes jobs (LLM inference, HTTP fetches, custom compute) off-chain. Runners stake CBY (self-bonded or via delegated stake from CBY holders, see CIP-13) and are selected via VRF. Results are verified under developer-selected trust models: N-of-M quorum, economic bond, TEE attestation, structured matching, semantic similarity, or (in v2) ZK-proofs. A commit-reveal protocol with a 15-minute challenge window and slashing enforces honesty.
- **Dual-Metered Gas.** Two independent fee markets price computation (**Cycles**) and data (**Cells**) separately. Each meter uses a basefee that adjusts dynamically with demand and

is burned; tips go to proposers. This prevents cross-subsidization between compute-heavy and storage-heavy workloads.

3.3 Differences vs. Ethereum

Aspect	Ethereum	Cowboy
Execution	EVM bytecode (Solidity)	Python actors in sandboxed PVM
Fees	Single gas scalar	Dual meters (Cycles + Cells)
Scheduling	External keepers required	Native timers with GBA auction
Off-chain compute	External oracles	Verifiable Runner marketplace
State management	Indefinite storage	Rent with eviction and restoration

4 Accounts and State

Cowboy distinguishes two object types:

- **External Accounts (EOAs):** Controlled by private keys. They initiate transactions and hold balances of CBY and other assets. Key management may be abstracted via passkeys or other WebAuthn-compatible mechanisms in future protocol versions.
- **Actors:** Autonomous Python programs executed in the PVM. Actors own storage, receive messages, and can send messages to other actors.

Each object has a 20-byte address. Actor addresses are computed CREATE2-style from the creator address, a salt, and the code hash. The **world state** is a mapping:

State : `Address` $\rightarrow$ `{ balance, nonce, code_hash?, storage?, metadata }`

where actor storage is a key/value map subject to quotas and rent. System actors and precompiles occupy a reserved prefix of the address space (0x0000...0100).

5 Transactions and Message Passing

A user interacts with Cowboy by sending a signed **transaction** specifying a destination, a payload, and resource limits: a **cycles limit** and a **cells limit** alongside maximum and tip prices for each.

An actor interacts with other actors by sending **messages**. Messages carry a payload, may transfer value, and may trigger further messages. Delivery is **exactly-once after finality**; before finality it is **at-least-once** and may be reverted. Handlers **MUST** be idempotent. Actors may schedule **timers** that insert messages at a future block height. To avoid denial-of-service through explosive fanout, a transaction (including all nested sends) **MUST NOT** enqueue more than **1,024** messages.

5.1 Native Timers and the Actor Scheduler

Cowboy provides protocol-native timers, eliminating the need for external keeper networks. Actors schedule messages to themselves or other actors at a future block height or on a recurring interval.

Tiered Calendar Queue. The scheduler uses three tiers to manage timers across different time horizons with constant per-block cost:

- **Tier 1 — Block Ring Buffer:** Imminent timers, one slot per block. $O(1)$ enqueue/dequeue.
- **Tier 2 — Epoch Queue:** Medium-term timers, migrated in batches to Tier 1 at epoch boundaries. $O(1)$ amortized.
- **Tier 3 — Overflow Sorted Set:** Long-horizon timers in a Merkleized binary search tree. $O(\log n)$.

Gas Bidding Agent (GBA). Rather than pre-paying a fixed gas fee, an actor may designate a GBA — another actor that dynamically bids for timer execution on its behalf. Actors that do not specify a GBA receive a protocol-provided default that bids conservatively. When a timer becomes due, the protocol performs a read-only call to the GBA with a context containing current basefees, timer urgency (how many blocks deferred), and the owner's balance. The GBA returns a competitive bid, creating an intra-block auction for the timer lane's compute budget.

Fairness and Liveness. Deferred timers receive a weighted priority boost with exponential decay, preventing perpetual starvation by high-bidding actors.

DoS Prevention. Multiple layers protect the timer system:

Attack Vector	Mitigation
Schedule millions of timers	Progressive deposit: $\text{deposit}(n) = \text{base_deposit} \times (1 + \text{floor}(n / 100))$
Sybil attack across many actors	Per-block execution budget caps total timer work (20% of block cycles)
Timer bomb (many timers, one block)	Exponential same-block surcharge: $\text{surcharge}(k) = \text{base_cost} \times 2^{\max(0, k - 16)}$
Fill queue far in advance	Timer basefee rises automatically as queue depth exceeds target
Outbid everyone perpetually	Anti-starvation boost for deferred timers
DoS then cancel for refund	Surcharges are burned, not refunded on cancellation

Per-actor timer limit: **1,024** active timers (enforced via per-actor secondary index). Deposits are refunded when a timer fires or is cancelled.

6 Asynchronous Execution and Multi-Block Semantics

6.1 Single-Block Atomicity

Cowboy provides **atomicity only within a single block**. All state reads, writes, and outbound messages within a handler are atomic — they either all commit or all revert. There is no cross-block atomicity. Once a handler completes and the block is finalized, subsequent handlers execute in potentially different world state.

6.2 Why No Cross-Block Transactions

Cross-block atomicity would require either global locks (destroying parallelism and creating deadlock vectors) or speculative execution with rollbacks (creating grieving opportunities and unpredictable costs). Cowboy explicitly rejects both approaches.

The fundamental problems with suspending execution across blocks are: (1) stale state — values read before the yield may have changed; (2) invalid control flow — branches taken on pre-yield state may no longer be appropriate; (3) composability explosion — nested yields create interleavings where each path depends on invalidated assumptions; (4) adversarial grieving — attackers can mutate state between yield points.

6.3 Message-Passing Continuation Model

Instead of implicit continuations, Cowboy uses **explicit message passing** for all asynchronous operations. When an actor dispatches an off-chain job, the result arrives as a separate message in a later block. The actor must re-read and re-validate any state assumptions in the callback handler.

This model requires **explicit context capture** — any state needed in the continuation must be included in the outbound message. Correlation IDs enable matching responses to requests. If an actor sends multiple requests, responses may arrive in any order.

Actors **MUST** implement timeout handling for operations that depend on external responses. The recommended pattern combines correlation tracking with native timers: schedule a timeout timer alongside each outbound request, and cancel the timer when the response arrives.

Model	Atomicity	Developer	
		Burden	Griefing Resistance
Ethereum (sync calls)	Single TX	Low	High
Cross-block locks	Multi-block	Low	Low (deadlocks, lock griefing)
Optimistic + rollback	Multi-block	Medium	Low (rollback spam)
Cowboy (message passing)	Single block	Medium	High

7 The Cowboy Actor VM (PVM)

Cowboy uses Python as its execution language. The PVM executes Python bytecode in a deterministic sandbox pinned to Python 3.11.8, with restrictions to ensure identical results across all nodes.

7.1 Determinism Guarantees

Runtime Environment:

- No JIT compilation — pure interpretation mode only.
- Deterministic memory management via reference counting. The cyclic garbage collector is disabled; creating reference cycles **MUST** raise `DeterminismError`.

- Fixed recursion limit: **32** (integrated with cycle metering).

Numeric Determinism:

- All floating-point operations use a cross-platform software math library (softfloat), not the host FPU.
- Transcendental functions (`sin`, `cos`, `log`, `exp`, etc.) use deterministic softfloat implementations.
- If the `decimal` module is used, it **MUST** use fixed rounding mode (`ROUND_HALF_EVEN`) and fixed precision.

Collection Ordering:

- `PYTHONHASHSEED` fixed to 0.
- `dict` iteration is insertion-ordered (deterministic per Python 3.7+).
- Built-in `set` and `frozenset` are replaced with `ordered_set` from `cowboy_sdk.collections` (transparent to user code).

Serialization:

- All data crossing trust boundaries **MUST** use canonical CBOR (RFC 8949, §4.2): sorted map keys, shortest integer encoding, no indefinite-length containers, 64-bit floats only.
- `pickle` is forbidden. JSON output **MUST** use `sort_keys=True`, `separators=(',', ':')`.

Module Whitelist (v1):

`collections`, `dataclasses`, `enum`, `functools`, `itertools`, `json`, `re`, `struct`, `math`, `decimal`, `typing`, `abc`, `hashlib`, `cowboy_sdk`. No C extensions. No dynamic imports. Additional modules may be added via governance after determinism audit.

Forbidden Operations:

Category	Forbidden
System	<code>sys.exit()</code> , <code>os.environ</code> , <code>os.system()</code> , <code>subprocess.*</code>
Time	<code>time.time()</code> , <code>datetime.now()</code> , <code>time.sleep()</code>
Randomness	<code>random.*</code> (use <code>cowboy_sdk.vrf</code> instead)
Networking	<code>socket.*</code> , <code>urllib.*</code> , <code>http.*</code> , <code>requests.*</code>
Filesystem	All except <code>/tmp</code> scratch space (256 KiB limit, wiped post-handler)
Reflection	<code>eval()</code> , <code>exec()</code> , <code>compile()</code> , <code>globals()</code> modification

Category	Forbidden
Introspection	<code>sys._getframe()</code> , <code>inspect.currentframe()</code> , <code>gc.*</code>
Weak References	<code>weakref.*</code> (non-deterministic collection timing)
Threading	<code>threading.*</code> , <code>multiprocessing.*</code> , <code>concurrent.*</code>
Identity	<code>is</code> comparisons on strings or numbers (use <code>==</code>)

7.2 Storage and State Persistence

The storage architecture uses a three-layer model (see CIP-4):

1. **The Ledger:** Append-only log of blocks — the sequential, historical source of truth.
2. **QMDB (canonical state):** Three flat key-value databases (`state_db`, `tx_index`, `tx_receipts`) with Blake3 Merkle commitments, generating a verifiable `state_root` per block. Fixed 54-byte keys with byte-prefix routing provide $O(1)$ reads and writes without tree rebalancing. Blake3 hashing is 3–5× faster than Keccak-256.
3. **Auxiliary Indexes:** Rebuildable, read-optimized tables for transaction hashes, event topics, etc. Not part of the consensus-critical state root.

Light Client Verification. QMDB provides native Merkle inclusion proofs for any state key. A standalone `cowboy-proof-verifier` crate (Rust + WASM) enables verification with only the proof data and state root — no full node required.

All actor storage is subject to **state rent** (see §4.4 and the Data Availability section below).

8 Pricing: Cycles and Cells

Ethereum uses a single gas scalar. Cowboy splits pricing into two independent meters:

- **Cycles** measure compute: Python operations and host calls each have a fixed cost defined in a consensus-critical cost table. Cycles resemble Erlang reductions — a budget of discrete steps bounding handler execution.
- **Cells** measure bytes: `calldata`, `return data`, `blobs`, and `storage writes` all consume cells (1 cell = 1 byte).

Each block adjusts two basefees (one per meter) dynamically based on demand. Users specify max prices and optional tips for each meter. Basefees are burned; tips go to proposers.

The protocol does **not** meter off-chain Runner execution. Runners set their own prices in a free market, and actors escrow CBY at submission. This separates deterministic on-chain metering from non-deterministic off-chain resource pricing.

9 Off-Chain Compute: The Runner Marketplace

Actors can outsource computation — LLM inference, HTTP fetches, heavy transforms — to a decentralized network of **Runners** who stake CBY. Runners may self-bond their own CBY or accept **delegated stake** from third-party CBY holders who earn a runner-configured share of settlement revenue (CIP-13). The marketplace is verifiable: the chain accepts results under trust models chosen by the developer, and dishonest runners risk slashing of both self-bonded and delegated stake.

Job Lifecycle (CIP-2 §8 — Commit-Reveal with Designated Aggregator):

1. **Post:** Actor submits a job with escrowed price, resource bounds, and trust model.
2. **Assign:** A committee of M runners is selected via VRF from eligible (staked, healthy) runners.
3. **Commit:** All M runners independently execute the job and submit `hash(result || sig)` on-chain.
4. **Collect:** The designated aggregator (highest-reputation runner in the committee) collects plaintext results from other runners off-chain.
5. **Reveal:** Aggregator submits the aggregated `VerifiedResult` plus runner signatures on-chain.
6. **Verify:** Result Verifier (0x03) validates hash matching against stored commitments, verifies runner signatures, and executes the chosen verification mode (MajorityVote, Structured-Match, Deterministic, etc.).
7. **Payout:** 89% of job payment to runner(s), 10% burned, 1% to Treasury. Aggregator receives a small bonus for honest aggregation.

Trust Models:

Mode	Level of Trust
N-of-M Quorum	Committee executes; runtime accepts consensus result.
N-of-M with Dispute	Runners stake a bond; disputers may prove incorrect results within a fixed window.

Mode	Level of Trust
TEE Attestation	Execution within a Trusted Execution Environment; attestation verified on-chain.
ZK-Proof (v2)	Runners provide zk-SNARKs for cryptographic verification.

Runners and actors are matched by **Entitlements** — a declarative permissions framework governing capabilities such as TEE requirements, data residency, and supported models (see §15).

9.1 Runner Resource Accounting

Every job submission includes explicit resource bounds (max tokens, max wall time, max memory, max price). Runners publish rate cards to an on-chain registry. Job pricing follows: $\text{actual_payment} = \min(\text{reported_usage} \times \text{rates}, \text{max_price})$. Tips incentivize priority during high demand.

Runner-reported usage is trusted by default, subject to reputation scoring and anomaly detection ($>2\times$ expected usage triggers automatic review). For stronger guarantees, actors can require TEE-attested metering. Proven dishonesty (e.g., fabricated results, misreported usage) is subject to stake slashing via the challenge mechanism; operational failures (timeouts, crashes) result in reputation penalties only.

Payment and Failure Handling:

Outcome	Runner Payment	Actor Refund
Success	89% of $\min(\text{reported_usage} \times \text{rates}, \text{max_price}) + \text{tip}$ (10% burned, 1% Treasury)	$\text{max_price} - \text{actual_payment}$
Runner fault (timeout, invalid result, crash)	0	100% of escrow
Impossible job (bounds too tight)	Pro-rata based on progress	Remainder of escrow
Actor fault (malformed input)	Minimum fee (gas cost recovery)	Remainder of escrow
External fault (API down, model unavailable)	Pro-rata based on progress	Remainder of escrow

9.2 LLM Result Verification

LLM outputs are inherently non-deterministic — the same prompt can produce semantically equivalent but byte-different outputs. Cowboy provides multiple verification modes:

Mode	Run-ners	Verification	Challenge Scope	Use Case
<code>none</code>	1	None	Non-delivery only	Prototyping, low-stakes
<code>economic_bond</code>	1	Objective checks	Objective failures	Subjective generation
<code>majority_vote</code>	N-of-M	Vote on field value	Objective failures	Classification
<code>structured_match</code>	N-of-M	Verifier functions	Objective failures	Structured extraction
<code>deterministic</code>	N-of-M	Exact match + TEE	Full reproduction	Critical deterministic
<code>semantic_similarity</code>	N-of-M	Embedding threshold	Objective failures	Subjective with similarity

These modes provide **economic assurance**, not cryptographic correctness, except where TEE or ZK verification is explicitly required. The protocol guarantees execution integrity, not output quality — quality is a market outcome.

Objective Failure Criteria (automatically detected, no challenge required):

Failure	Detection	Consequence
Schema violation	Output fails declared JSON schema	Reputation penalty, no payment
Timeout	No result within <code>max_wall_time</code>	Reputation penalty, no payment
Empty/garbage output	Output below <code>min_length</code> or fails entropy check	Reputation penalty, no payment

Failure	Detection	Consequence
Non-delivery	Runner accepted job but never submitted	Reputation penalty, no payment
Wrong model	TEE attestation shows different model hash	Slash (proven dishonesty)
Prompt injection leak	Output contains system prompt markers	Reputation penalty, no payment

Only proven dishonesty (fabricated results, wrong model with TEE attestation) triggers stake slashing. Operational failures reduce the runner’s reputation score, affecting future job selection, but do not destroy stake.

9.3 External Data and Oracle Semantics

Actors frequently need external data: price feeds, web APIs, public datasets. External data is inherently mutable and non-deterministic. Different sources require different verification strategies:

Source Type	Characteristics	Verification Strategy
Deterministic API	Versioned, stable, structured (blockchain RPC, static files)	Exact match
Semi-stable API	Structured with variable metadata (REST APIs)	Structured match, ignore metadata
Time-series data	Values change over time (price feeds)	Median/majority with freshness bounds
Web scraping	Unstructured, highly variable (HTML pages)	Extraction-based matching
Authenticated endpoints	Requires credentials	Single runner + TEE

Freshness. Actors specify freshness constraints with a reference mode (`block`, `submission`, or `absolute`) and a `max_age_seconds` parameter.

Snapshot Modes. When multiple runners fetch mutable data, the protocol selects a canonical result via one of: **first_valid** (first runner's result is authoritative), **median** (for numeric data), **majority** (for categorical data), or **latest** (most recent by timestamp).

Extraction-Based Verification. For web scraping, runners apply extraction rules (CSS selectors, XPath, JSONPath, regex) and submit extracted data rather than raw HTML. Verification compares extracted fields, ignoring irrelevant page differences.

HTTP access is governed by the Entitlements system. Actors declare required domains; runners advertise supported domains. The protocol provides curated domain sets (**price_feeds**, **government_us**, **social_apis**, **blockchain_rpc**).

10 Randomness

Each block derives a random beacon from the previous quorum certificate using a threshold BLS VRF. Actors access sub-randomness via `HKDF(R_n, label)` for fair committee sampling, lotteries, and games.

11 Consensus and Networking

Cowboy uses **Simplex BFT** with proof-of-stake. Key parameters: ~1-second blocks, ~2-second finality, fault tolerance up to $f < n/3$ Byzantine validators.

Consensus Flow:

1. **Propose:** Current proposer (selected by VRF, weighted by stake) broadcasts a block.
2. **Vote:** Validators vote; signatures are buffered and batch-verified at quorum ($2f+1$).
3. **Certify:** A Quorum Certificate (QC) is formed from $2f+1$ votes.
4. **Finalize:** A block is final when its direct child has a QC (two-chain commit).

Under partial synchrony, the protocol guarantees safety at all times and liveness when network delay is bounded. If a proposer fails, validators execute a view change: broadcast their highest QC, and the next VRF-selected leader proposes extending the highest-QC block.

11.1 Validator Set

The validator set is open and permissionless. Requirements: stake $\geq$ `minimum_validator_stake` (governance-tunable), compliant validator software. No protocol cap on validator count. Validator staking is self-bonded only; runner staking supports delegation (CIP-13).

Lifecycle: Register (stake CBY, submit BLS12-381 key) → Activate (next epoch boundary) → Operate (propose, vote, earn rewards) → Exit (signal unbonding) → Withdraw (after 7-day unbonding period).

Epochs: 3600 blocks (~1 hour). Validator set updates, slashing penalties, and epoch randomness derivation occur at epoch boundaries.

11.2 Staking and Rewards

Block rewards (from inflation) are distributed proportionally to stake. Proposers additionally receive transaction tips. Validator staking is self-bonded. Runner staking supports delegation: CBY holders may delegate stake to runners and earn a runner-configured share of the 89% settlement payout (CIP-13). Delegated stake increases a runner's VRF selection weight and maximum job capacity.

11.3 Slashing

Cowboy uses a conservative slashing model — most offenses result in jailing (temporary removal) rather than stake destruction:

Offense	Penalty
Double signing	Jail + slash 1% of stake
Proposer equivocation	Jail + slash 1% of stake
Extended downtime (>50% of votes over 1000 blocks)	Jail (no slash)
Invalid block proposal	Jail (no slash)

Jailed validators must wait 24 hours before unjailing. Repeated offenses increase jail duration exponentially.

11.4 Network Layer

Transport: QUIC over TLS 1.3 (required). Gossip: transactions flood to all peers; blocks are relayed by validators; votes go directly to the proposer. Peer discovery: DHT-based with bootstrap nodes.

11.5 Dedicated Lanes

Block space is partitioned into dedicated lanes with reserved capacity:

Lane	Reserved Capacity	Contents
System	5%	Validator updates, governance, slashing
Timer	20%	Scheduled timer executions
Runner	25%	Runner job results and attestations
User	50%	User-initiated transactions

Unused capacity in higher-priority lanes cascades to lower-priority lanes. Transactions are tagged by type at submission; the proposer cannot reassign lanes. Each lane has independent fee multipliers applied to the global basefees.

11.6 MEV Reduction

Cowboy mitigates MEV through four mechanisms: (1) mandatory per-block proposer rotation via VRF prevents multi-block observation; (2) VRF-based transaction ordering within blocks prevents strategic placement; (3) ~2-second finality minimizes the observation window; (4) lane isolation prevents congestion attacks that delay victim transactions. No encrypted mempool is used — the marginal benefit does not justify the added latency given the already-minimal MEV surface. This does not prevent proposer inclusion/censorship or private orderflow MEV.

12 Data Availability, State Rent, and Storage

12.1 Inline Data vs. Blobs

Small outputs (≤ 64 KiB) are stored inline and paid for with cells. Larger artifacts **MUST** be stored as content-addressed blobs (e.g., IPFS) with the multihash referenced on-chain.

12.2 State Rent

All persistent actor storage is subject to **state rent** — an ongoing fee for occupying space in the global state trie. Rent rates adjust dynamically based on total network state size:

$$\text{rent_rate}_{\{i+1\}} = \text{rent_rate}_i \times (1 + \text{clamp}((S - T) / (T \times \alpha), -\delta, +\delta))$$

where S is the current total state size, T is the target (governance-tunable), $\alpha = 8$, and $\delta = 0.125$.

Rent is auto-deducted from the actor's balance each rent epoch (default: 1 day). Actors may also prepay rent for cost certainty, and any account may sponsor rent on behalf of any actor. Each actor maintains a minimum balance reserve (~5 weeks of rent) as a buffer before entering grace period.

12.3 Grace Period and Eviction

When an actor cannot pay rent (balance, reserve, and prepaid epochs exhausted):

- **Grace period (7 rent-epochs):** Actor remains fully functional; flagged as “rent overdue”; catch-up fee accumulates (10% of missed rent).
- **Warning period (3 rent-epochs):** Actor flagged as “eviction imminent”; events emitted to alert dependent actors.
- **Eviction (rent-epoch $N+11$):** Actor storage and active timers are pruned. Code, address, balance, and storage root hash are preserved. The actor enters a “dormant” state.

Evicted storage can be restored by anyone who provides the original data (verified against the recorded root hash) and pays back-rent plus catch-up fees.

12.4 Storage Quotas

Each actor has a base storage quota of **1 MiB**, extendable up to **8 MiB** via a storage bond. The bond is locked while the quota is in use, returned when reduced, and forfeited if the actor is evicted. Rent applies to the full allocated quota.

13 The State Transition Function

The state transition function takes a block and an input state and returns the next state. Let σ be the global state, B a block with transactions T_i , basefees (bf_c , bf_b), and randomness R .

1. **Header/Proposer:** Determined by Simplex; R derives from the parent QC.

2. **Execute Transactions:** Per-lane selection, then VRF ordering. For each transaction: validate signature, nonce, and balance; initialize meters; dispatch to target (fanout $\leq 1,024$, reentrancy depth ≤ 32); enforce memory (10 MiB), mailbox ($\leq 1,000,000$ bytes), and storage quotas; deduct fees and burn basefees.
3. **Deliver Timers:** Inject due timers at height(B).
4. **Resolve Jobs:** Process commitments, reveals, challenges, and payouts.
5. **Adjust Basefees:** Update (bf_c, bf_b) based on block utilization.
6. **Mint Rewards:** Distribute per-block inflation to validators.

14 Normative Conventions

This document uses **MUST/SHOULD/MAY** as defined in RFC 2119. Parameters marked **governance-tunable** can be changed by on-chain governance (see §11).

15 1. Accounts, Addresses, and Keys

15.1 1.1. Signatures

External accounts **MUST** use **secp256k1** (ECDSA) with chain-id separation.

15.2 1.2. Actor address derivation (CREATE2-style)

New actor addresses **MUST** be: `addr = last_20_bytes(keccak256(creator || salt || code_hash))` where `code_hash = keccak256(python_source_bytes)`.

`python_source_bytes` **MUST** be canonicalized as UTF-8, NFC-normalized, LF line endings, and no BOM. The canonical bytes are what are hashed and stored.

15.3 1.3. System address space

The range 0x0000...0100 is reserved for **system actors and precompiles** (see §10).

16 2. Transaction Types & Encoding

16.1 2.1. Typed tx (EIP-1559 style, dual meters)

A transaction MUST include: `chain_id`, `nonce`, `to`, `value`, `cycles_limit`, `cells_limit`, `max_fee_per_cycle`, `max_fee_per_cell`, `tip_per_cycle`, `tip_per_cell`, `access_list?`, `payload`, `signature`.

16.2 2.2. Validity checks

Nodes MUST reject a tx if: (a) limits exceed maxima (§13.1), (b) insufficient balance, (c) signature invalid, (d) access list invalid, or (e) payload decoding fails.

16.3 2.3. Fee accounting

Let `bc`, `bb` be the block basefees for cycles/cells. Fees are: `fee = cycles_used * (bc + min(tip_per_cycle, max_fee_per_cycle - bc)) + cells_used * (bb + min(tip_per_cell, max_fee_per_cell - bb))`. Unused limits MUST be refunded at the user's `max_fee_*` rates.

16.4 2.4. EBNF (informative)

`Tx = Header Body Sig` `Header = chain_id nonce to value cycles_limit cells_limit max_fee_per_cycle max_fee_per_cell tip_per_cycle tip_per_cell [access_list]`
`Body = payload Sig = secp256k1_signature_recoverable`

16.5 2.5. Encoding (normative)

Transactions MUST be encoded as canonical CBOR (RFC 8949, deterministic encoding) arrays in fixed field order:

`Tx = [chain_id, nonce, to, value, cycles_limit, cells_limit, max_fee_per_cycle, max_fee_per_cell, tip_per_cycle, tip_per_cell, access_list, payload, signature]`

- `to` is a 20-byte address or `null` for actor creation.
- `access_list` is either `null` or a list of `[address, storage_keys[]]` pairs.
- `signature = [y_parity, r, s]` where `y_parity` $\in \{0,1\}$ and `r,s` are 32-byte big-endian byte strings.

- The signing hash MUST be `keccak256(CBOR(Tx_without_signature))`, where `Tx_without_signature` is the same array with the `signature` field set to `null`.

17 3. Execution Model (Actors)

17.1 3.1. Runtime & Determinism

- Official SDKs: **Python SDK**. The runtime MUST enforce determinism:
 - **Allowed operations:** Standard Python operations, file I/O limited to `/tmp`; `async/await` is syntax sugar only and does not suspend across blocks.
 - **Forbidden:** `sys.exit()`, `random` module (except chain VRF), `time.time()/datetime.now()`, `os.environ` access, socket/network operations, subprocess calls, path traversal outside `/tmp`.
 - **Floating point:** Permitted; Cowboy provides a deterministic math library.
 - **Scratch space:** `/tmp` MUST be per-invocation, capped at **256 KiB** (counts toward `cells_used`), wiped post-handler.

17.2 3.2. Memory & Storage

- **Per-call memory limit:** 10 MiB heap memory.
- **Per-actor persistent storage quota:** 1 MiB (governance-tunable) with **state rent** (§4.4).
- **Quota extensions:** An actor MAY post a **storage bond** up to 8 MiB total; rent applies to the full allocated quota.

17.3 3.3. Messaging, Reentrancy, Timers

- **Delivery: Exactly-once after finality** (before finality: **at-least-once**). Each message ID MUST be `keccak256(sender||nonce|msg_hash)` and recorded in a per-actor dedup set (counts toward actor storage or a separate metered mailbox store). Dedup entries MUST be retained for at least `dedup_window` after finality.
- **Mailbox:** Capacity **1,000,000 bytes** (or equivalent cell-metered limit); enqueue beyond the limit MUST revert.
- **Per-tx fanout:** A transaction (including all nested sends) MUST NOT enqueue more than **1,024** messages.
- **Reentrancy:** Allowed within a single block only; there is no synchronous call/return across blocks. **Recursion/await depth cap = 32**.

- **Timers (chain-native):** The following timer primitives are provided:
 - `timer_id = set_timer(height, handler, data)` — Schedule a one-time timer for the specified block height. Returns a unique `timer_id`.
 - `timer_id = set_interval(every_n_blocks, handler, data)` — Schedule a recurring timer. Returns a unique `timer_id`.
 - `cancel_timer(timer_id)` — Cancel a pending timer by its ID. Returns the deposit if successful.
 - Timer delivery is **best-effort**; execution depends on the GBA auction (see §Timer Rate Limiting).

17.4 3.4. Randomness

- Validators **MUST** generate a threshold-BLS VRF per block: `R_n = VRF_sk_epoch(QC_{n-1})`. Actors **MAY** call an API which returns `HKDF(R_n, label)`.

17.5 3.5. Partial cost table (informative)

- Exact metering is consensus-critical; implementers **MUST** match the reference cost table.

Primitive	Cycles
Python arithmetic ops	1
Python function call	10
Dictionary get/set	3
List append/access	2
String operations (per char)	1
host: mailbox send (per msg excl. payload)	80
host: timer set/cancel	200
host: blob commit (per KiB)	40

Note: Cells meter bytes (payload, return data, inline blobs ≤ 64 KiB, /tmp).

18 4. Fees, Metering, and Basefee Adjustment

18.1 4.1. Meters

- **Cycles:** Deterministic step count over Python operations + host calls.
- **Cells:** Bytes used by calldata, return data, inline blobs (≤ 64 KiB), and /tmp.

18.2 4.2. Dual EIP-1559 basefees

Let U_c, T_c be cycles usage/target; U_b, T_b be cells usage/target. With elasticity $E=2$ (hard cap $E \cdot T_*$), adjustment uses:

```
basefee_{x,i+1} = max(1, basefee_{x,i} * (1 + clamp((U_x - T_x)/(T_x*alpha),
-delta, +delta)))
```

where $x \in \{\text{cycle}, \text{cell}\}$, $\alpha = 8$, $\delta = 0.125$. Nodes **MUST burn 100%** of basefees; tips go to proposers/validators.

18.3 4.3. Targets (genesis defaults)

- T_c (cycles target): **10,000,000 cycles** (cap **20,000,000**).
- T_b (cells target): **500,000 bytes** (cap **1,000,000**).

18.4 4.4. State rent

Persistent storage incurs **rent** per byte per rent-epoch, which is governance-tunable. If unpaid for **7 rent-epochs**, an eviction warning begins; eviction is eligible after **10 rent-epochs**.

19 5. Off-Chain Compute

19.1 5.1. Model registry

`model_id = keccak256(weights||arch||tokenizer||license)` **MUST** uniquely identify a model revision. Publishing is permissionless with a **refundable 1,000 CBY deposit**. Governance **MAY** flag/ban models.

19.2 5.2. Runner staking & delegation

Runners MUST maintain an **effective stake** of $\max(10,000 \text{ CBY}, 1.5 \times \text{declared_max_job_value})$ in the Runner Registry. Effective stake = self-bonded stake + delegated stake from third parties.

Self-bond: Runners deposit their own CBY directly. A minimum self-bond ratio (`MIN_SELF_BOND_BPS`, default 10% of effective stake) ensures the runner always has skin in the game.

Delegation (CIP-13): CBY holders may delegate stake to a runner via the `RunnerDelegateStake` system instruction. Delegated stake increases the runner's VRF selection weight and maximum job value. In return, delegators receive a share of the runner's 89% settlement payout, configured by the runner's `commission_bps` (runner's cut of delegator-attributed revenue). Delegators initiate withdrawal via `RunnerUndelegateStake`; funds are released after an unbonding period (`UNBONDING_BLOCKS`, default ~24 hours). Delegated stake is slashable proportionally alongside the runner's self-bond.

19.3 5.3. Job lifecycle (commit-reveal with designated aggregator)

1. **Post:** Actor posts a job with escrowed price, resource bounds, and trust model.
2. **Assign:** A committee of **M** runners is sampled via VRF from eligible (staked, healthy) runners. The highest-reputation runner in the committee is designated the **aggregator**.
3. **Commit:** All **M** runners independently execute the job and submit `hash(result || sig)` on-chain within `commit_deadline_blocks`.
4. **Collect:** The aggregator collects plaintext results from other runners off-chain via direct HTTP push.
5. **Reveal:** The aggregator submits the aggregated `VerifiedResult` plus runner signatures on-chain. If the aggregator fails within `aggregator_timeout_blocks`, other runners may submit individual reveals; the Result Verifier falls back to self-aggregation.
6. **Verify:** Result Verifier (`0x03`) validates hash matching against stored commitments, verifies runner signatures, and executes the chosen verification mode. Proven dishonesty $\Rightarrow$ stake slashing. Operational failures (timeouts, crashes) result in reputation penalties only.
7. **Payout:** On finalization, **89%** of job payment to runner(s), **10%** burned, **1%** to Treasury. Aggregator receives a small bonus for honest aggregation.

19.4 5.4. Determinism & bounds

Jobs MUST pin `toolchain_digest` and `seed`. On-chain return data MUST be $\leq$ **64 KiB**.

19.5 5.5. TEE option

A job MAY set `tee_required=true`. A valid attestation MUST match accepted policies.

20 6. Consensus, Randomness, and Networking

20.1 6.1. Consensus

Simplex BFT PoS; ~1s target block time; **finality on commit** (~2s). Proposers rotate every block using the VRF beacon (mandatory rotation for MEV resistance). Votes aggregate via **BLS12-381** with buffered batch verification.

Message types (normative): PROPOSE, VOTE, NEW_VIEW.

Quorum certificate (QC): QC = {block_hash, height, round, aggregated_signature, signer_bitmap} where aggregated_signature is BLS12-381 over the block_hash || height || round domain.

VRF: Each block header MUST include the proposer’s VRF output and proof for the current height; validators MUST verify it against the epoch seed.

Finality rule: A block B_h is final when its direct child has a QC (two-chain commit). Concretely, if QC(B_{h+1}) exists and B_h is the parent of B_{h+1}, then B_h is finalized. Implementations MAY expose both “commit” and “finalized” states explicitly.

View change: On timeout, validators broadcast NEW_VIEW containing their highest known QC. The next proposer MUST build on the highest-QC block.

20.2 6.2. P2P transport

Implementations MUST support QUIC over TLS 1.3.

20.3 6.3. Dedicated Lanes

Block space is partitioned into **dedicated lanes** with reserved capacity:

Lane	Reserved Capacity	Priority	Contents
System	5%	Highest	Validator updates, governance, slashing
Timer	20%	High	Scheduled timer executions

Lane	Reserved Capacity	Priority	Contents
Runner	25%	High	Runner job results and attestations
User	50%	Normal	User transactions

Lane guarantees:

- Timer and runner lanes prevent user transaction spam from blocking autonomous actor execution
- Unused capacity in higher-priority lanes cascades to lower-priority lanes
- Each lane has independent fee multipliers applied to the global cycle/cell basefees

20.4 6.4. Gossip (mempool)

Public mempool prioritized by effective fee. Within each lane, proposers select transactions by highest effective fee up to lane capacity (ties broken by `tx_hash`), then apply VRF ordering within the selected set. For ordering, `effective_fee` is computed using **intrinsic** costs only:

$$\text{effective_fee} = \text{intrinsic_cycles} \times \min(\text{max_fee_per_cycle}, \text{basefee_cycle} + \text{tip_per_cycle}) + \text{intrinsic_cells} \times \min(\text{max_fee_per_cell}, \text{basefee_cell} + \text{tip_per_cell}).$$

Transactions are tagged by lane type. No private builders or encrypted mempool in v1—MEV resistance relies on fast finality and mandatory proposer rotation.

20.5 6.5. MEV Reduction (Limitations Apply)

Cowboy’s MEV mitigation strategy combines multiple mechanisms:

Mandatory proposer rotation: Simplex consensus rotates proposers every block via VRF. Unlike stable-leader protocols, no single validator can observe transaction flow across multiple blocks, limiting MEV extraction windows.

VRF-based transaction ordering: Within each block, transactions are ordered by:

```
order_key = VRF(proposer_key, tx_hash, block_height)
```

This deterministic-but-unpredictable ordering prevents proposers from strategically placing their own transactions.

Note: This does not prevent proposer inclusion/censorship or private orderflow MEV.

Fast finality: ~2 second finality (2 Simplex rounds) minimizes the window for:

- Front-running (limited observation time)
- Sandwich attacks (high risk of failed execution)
- Time-bandit attacks (chain never reorgs past finality)

Dedicated lanes: Reserved capacity for timers and runners ensures autonomous actors execute reliably regardless of user mempool congestion. Attackers cannot spam the user lane to delay victim transactions in other lanes.

No encrypted mempool: Commit-reveal schemes add latency and complexity. Given ~1s blocks and ~2s finality, the observation window is already minimal. The combination of VRF ordering + rotation + fast finality provides sufficient MEV resistance without the latency cost of encryption.

21 7. Data Availability & Blobs

21.1 7.1. Inline cap

Inline blob cap is **64 KiB** per output.

21.2 7.2. External blobs

Larger data **MUST** be **content-addressed** (e.g., IPFS). The on-chain commitment **MUST** be a multihash.

22 8. Economics, Inflation, and Fees

22.1 8.1. Ticker & supply

CBY. Genesis supply **1,000,000,000 CBY**.

22.2 8.2. Inflation

A decreasing inflation schedule is used to bootstrap network security. Gross inflation is offset by protocol burn mechanisms (§8.4); net inflation depends on network usage.

Phase	Years	Gross Inflation	Rationale
Bootstrap	1–2	8% / 6%	Attract validators and runners while fee revenue is immature
Glidepath	3–4	4% / 3%	Reduce dilution as fee revenue and network stability improve
Steady-state	5+	2% floor	Maintain security budget with low long-run dilution

Governance MAY adjust the inflation rate within the following guardrails: (a) hard cap of 10% gross annual inflation, (b) maximum annual change of 2 percentage points, (c) timelock + supermajority for any change. A security-floor trigger MAY temporarily increase inflation if the staked ratio falls below a governance-defined threshold; the increase auto-expires when the threshold is recovered.

22.3 8.3. Distribution at genesis

Genesis supply is split into a **Company Reserve** and a **Network Distribution** pool.

Company Reserve (66.67% — 666,700,000 CBY). Tokens held by or on behalf of the company, satisfying all outstanding token warrant obligations. Sub-buckets:

Bucket	% of Total	CBY	Vesting
Team & Core Contributors	20.00%	200,000,000	1-year cliff, 4-year linear monthly
Investors (Warrants / Token Rights)	20.00%	200,000,000	10% at TGE, 6-month cliff, 24-month linear
Advisors	3.33%	33,300,000	6-month cliff, 24-month linear, milestone gates
Treasury / Governance	13.34%	133,400,000	5% seed liquidity at TGE, remainder governance-voted

Bucket	% of Total	CBY	Vesting
Foundation / Ops Reserve	10.00%	100,000,000	5% at TGE, quarterly board-approved releases over 36 months

Network Distribution (33.33% — 333,300,000 CBY). Tokens distributed to network participants via protocol emission. These sit outside the Company Reserve.

Bucket	% of Total	CBY	Emission Model
Validator Rewards	18.00%	180,000,000	Decreasing emission over 60 months
Runner Compute Incentives	8.00%	80,000,000	Usage-based; paid per runner-hour
Developer Grants	3.00%	30,000,000	Milestone-based, grant committee
Liquidity Mining / DEX	2.33%	23,300,000	Front-loaded over 12 months
Community / Airdrops	2.00%	20,000,000	2 drops (TGE + 6 months)

22.4 8.4. Fee sinks & splits

Source	Split	Destination
Cycle & Cell basefees	100%	Burned
Cycle & Cell tips	100%	Proposer / validators
Runner job payments	89%	Runner(s) and their delegators (CIP-13)
Runner job payments	10%	Burned
Runner job payments	1%	Treasury
Slashed stake	100%	Burned

The runner fee burn is the primary deflationary mechanism beyond basefee burns. As network usage grows, the burn rate increases proportionally, creating a reflexive supply-reduction loop

that offsets inflation (see §8.2). Governance MAY adjust the runner fee burn percentage via standard proposal + timelock.

23 9. System Actors & Precompiles

Canonical system actor addresses (see `node/runner/src/system_actors.rs` for the authoritative assignment):

- **0x01 Runner Registry:** Runner registration, staking (self-bonded and delegated per CIP-13), capabilities, health, and reputation.
- **0x02 Job Dispatcher:** Job submission, VRF-based runner selection, job lifecycle management.
- **0x03 Result Verifier:** Commit-reveal aggregation, result verification, callback dispatch.
- **0x04 Secrets Manager:** Encrypted secret storage and TEE-gated secret release.
- **0x05 TEE Verifier:** Remote attestation verification for TEE-based runners.
- **0x06 Dual Basefee:** Protocol dual-metered basefee state storage (§4, CIP-3). Stream Key Manager responsibilities (CIP-7) are co-located here in v1.
- **0x07 Entitlement Registry:** Permission management, Runner Pool access control, and general RBAC (see CIP-2 §7).
- **0x08 Treasury:** Protocol fee accumulation; disbursement mechanism governed by §11 and CIP-12.
- **0x09 Governance:** On-chain governance state: proposals, parameter store, system actor version pointers, Security Council registry, and the upgrade mechanism. See CIP-12.
- **0x0A Storage Manager:** Volume metadata, `StorageCommitment` records, manifest roots, and storage-fee settlement (CIP-9).
- **0x0B Relay Registry:** Relay Node registration, stake, health, capacity, and repair coordination (CIP-9).

Governance-tunable parameters are read from `0x09.params` rather than hard-coded in individual actors; see CIP-12 §4.2 for the config consumer pattern.

24 10. Developer Experience (DX)

- **SDKs:** A primary Python SDK (`cowboy-py`) is provided.
- **Local dev:** A suite of tools including a single-node devnet (`cowboyd`), runner simulator, faucet, and explorer will be available.
- **Best practices:** Reentrancy guards, capability-scoped handles, and idempotent message handling are encouraged via the SDK.

25 11. Governance & Upgrades

Governance is implemented as the `0x09 Governance` system actor. The full specification is **CIP-12**; this section summarizes the normative surface.

25.1 11.1. Model

- **Bicameral voting from day one.** Every proposal (except Tier 4 meta-governance) MUST pass both a **staked-CBY chamber** (economic weight) and a **validator chamber** (one-validator-one-vote operational weight). A proposal that passes the stake chamber but fails the validator chamber is rejected.
- **No time-based sunset.** There is no scheduled transition away from the initial governance configuration. Any reduction in the Security Council's role is itself a Tier 4 governance decision.
- **Vote weight source.** Only **staked CBY** carries vote weight; unstaked balances have zero voice. Validator delegators MAY override their validator's vote per proposal; absent an override, delegated stake votes with the validator. Runner-delegated CBY (CIP-13) has zero governance vote weight in v1; it is a third, non-voting staked category. Weights are snapshotted at the proposal's `snapshot_block`; stake in the unbonding queue at snapshot still counts for its pre-unbond weight.

25.2 11.2. Entities

Three distinct entities with separated powers:

- **Foundation:** A legal entity that holds the treasury address and runs off-chain operations. The Foundation has **no protocol authority**. It receives funds only when a treasury disbursement proposal passes governance. Foundation employees MAY serve as Security Council members individually, but the Foundation SHOULD hold at most **one** Council seat.
- **Security Council:** A **7-of-9 multisig** of named individuals (core developers, independent security researchers, validator operators, community-elected seats). Its only powers are: (a) cancel a queued proposal during its timelock (Tier 0–3 only), (b) trigger fast-track execution on a Tier 3 system actor upgrade in an emergency, (c) circuit-break a specific system actor with mandatory retroactive ratification within 7 days. The Security Council MUST NOT propose, vote, upgrade, or disburse treasury funds directly. Council membership is modifiable only via Tier 4 meta-governance.

- **Labs Multisig:** A separate multisig controlling only the governance portal frontend (static assets in CBFS + the portal gateway actor). It has **no** protocol authority. A compromised Labs Multisig can only serve a bad UI; users retain the ability to vote via CLI or third-party portals pointed at the same 0x09 state.

25.3 11.3. Proposal tiers and timelocks

Proposals are tiered by blast radius. The following are initial defaults; all values are themselves governance-tunable via Tier 4.

Tier	Name	Stake quorum	Stake approval	Validator majority	Time-lock	Fast-track
0	Parameter tuning	10%	> 50%	> 50%	3 days	No
1	Registry & whitelist	15%	> 50%	> 50%	5 days	No
2	Treasury & disbursement	20%	> 60%	> 50%	7 days	No
3	System actor upgrade	25%	> 66%	> 66%	7 days	Yes: 6h, 40% / > 75% / > 66%
4	Constitutional / meta	33%	> 75%	> 66%	14 days	No

Before formal voting, every proposal **MUST** pass an on-chain temperature check (non-binding signal vote, no quorum; duration scales with tier). Proposal deposits are refunded on reaching the voting stage and burned on failed temperature checks.

25.4 11.4. System actor upgrades

Hot-code upgrades to any system actor (0x01–0x0B) are executed natively by 0x09 via a Tier 3 proposal. The upgrade payload identifies the target, a content-addressed PVM bytecode hash resolved via CBFS, an optional migration function, an activation block, and a rollback deadline.

At `activation_block`, the prior version is copied to a rollback slot, the new version becomes active, and an optional migration function runs atomically in a system transaction (failure

auto-reverts the activation). Until `rollback_deadline`, a Tier 3 fast-track MAY revert to the rollback slot without re-validating the old bytecode.

Validators MUST pre-fetch new bytecode from CBFS during the timelock window to ensure zero-downtime activation. Upgrades to 0x09 itself are governed by Tier 4 meta-governance executed through the same mechanism.

25.5 11.5. Non-upgradeable via governance

The following MUST NOT be changed via 0x09 and require a validator-coordinated hard fork: block/transaction format, signature schemes, Simplex BFT consensus rules, and genesis allocations.

See **CIP-12** ([docs/cips/cip-12-governance.md](#)) for the full state layout, payload types, bicameral tally rules, Security Council pause semantics, and rationale.

26 12. Security Considerations

26.1 12.1. DoS limits (consensus-enforced; governance-tunable)

- `max_tx_size` = 128 KiB
- `max_message_depth_per_tx` = 32
- `per_actor_per_block_cycles` = 1,000,000 (burstable)

26.2 12.2. Runner safety

Slashing for proven dishonesty (fabricated results, wrong model). Operational failures result in reputation penalties only. Committees mitigate single-runner faults.

26.3 12.3. Reentrancy

Allowed but depth-capped; `stdlib` provides reentrancy guards.

26.4 12.4. Randomness bias

Threshold-BLS VRF with epoch keys; actors derive sub-randomness via HKDF.

26.5 12.5. State rent & eviction

Prevents state bloat; eviction windows protect liveness.

27 13. Parameters (Genesis Defaults)

Execution:

`memory_per_call = 10 MiB`; `storage_quota_per_actor = 1 MiB`; `reentrancy_depth = 32`; `fanout_per_tx = 1024`; `mailbox_capacity_bytes = 1,000,000`; `dedup_window = 10,000` blocks.

Fees:

`T_c = 10,000,000` cycles; `T_b = 500,000` bytes; `alpha = 8`; `delta = 0.125`.

Consensus:

`minimum_validator_stake = governance-tunable`; `epoch = 3600` blocks (~1 h); `block_time = 1` s; `finality = ~2` s; `unbonding_period = 7` days; `jail_period = 24` h; `double_sign_slash = 1%`; `consensus_protocol = Simplex BFT`.

Dedicated Lanes:

`system_lane_capacity = 5%`; `timer_lane_capacity = 20%`; `runner_lane_capacity = 25%`; `user_lane_capacity = 50%`.

Off-chain:

`committee M = 5`; `threshold N = 3`; `challenge_window = 15` min; `challenge_bond = 100` CBY; `runner_stake_floor = 10,000` CBY; `min_self_bond_bps = 1000` (10%); `unbonding_blocks = 7,200` (~24h); `max_delegators_per_runner = 200`; `max_active_tranches_per_delegator = 8`.

State Rent:

`target_state_size = governance-tunable`; `grace_period = 7` rent-epochs; `warning_period = 3` rent-epochs; `catch_up_fee = 10%`; `reserve_multiplier = 0.1`.

Economics:

`supply = 1,000,000,000`; `company_reserve = 66.67%`; inflation follows the schedule in §8.2; `basefee_burn = 100%`; `runner_fee_burn = 10%`; `job_fee_to_treasury = 1%`; `runner_payout = 89%`.

28 14. Differences vs. Ethereum

- **Execution:** Python actors vs. EVM contracts.
- **Fees:** Dual meters (cycles/cells) vs. single gas scalar.
- **Timers:** Native timers vs. external keepers.
- **Off-chain compute:** Native verifiable market vs. external oracles.
- **State:** Rent with eviction vs. indefinite storage.

29 15. Entitlements

A declarative, composable permissions system governs the capabilities of actors and runners. Entitlements control access to resources like networking, storage, and execution parameters, enforcing least-privilege by default. The system is enforced at deployment time, by the scheduler, and at the VM syscall gate.

29.1 15.1. Goals

- Least privilege by default.
- Deterministic enforcement.
- Declarative & composable.
- Auditable on-chain.

29.2 15.2. Objects & lifecycle

- **Actor Entitlements:** Permissions the actor requires.
- **Runner Entitlements:** Capabilities the runner provides.

29.3 15.3. Rules

1. MUST: Actors **require** entitlements; runners **provide** them.
2. MUST: Scheduler matches only if **requires** $\subseteq$ **provides**.
3. MUST: Syscalls fail if the corresponding entitlement is missing.
4. MUST: Child actors only inherit entitlements marked **inheritable:true**.

(For a full list of entitlements, see the [Entitlements Specification](#).)

30 16. Ethereum Interoperability

Cowboy's interoperability with Ethereum is a primary design goal, enabling seamless asset transfer and cross-chain communication. This is achieved through a combination of shared cryptographic primitives, a canonical bridge, and event subscription mechanisms.

30.1 16.1. Account Unification

- Cowboy external accounts (EOAs) MUST use the same `secp256k1` elliptic curve for signatures as Ethereum. This allows a single private key to control accounts on both networks, simplifying key management for users and agents.
- An actor, through a host call, MAY verify an EIP-712 signed data structure against a given Ethereum address, enabling actors to validate off-chain authorizations from Ethereum users.

30.2 16.2. Bridge Infrastructure

Cowboy relies on third-party bridge infrastructure for asset transfers and cross-chain message passing between Cowboy and Ethereum.

Requirements:

- The bridge MUST support the locking of native ETH and ERC-20 tokens on Ethereum to mint a corresponding wrapped representation on Cowboy (`wETH`, `wERC-20`), and the reverse burn-to-unlock flow.
- The bridge MUST support generic message passing: a transaction on one chain triggering a message call to a designated recipient on the other.
- Bridge selection and integration are determined by governance. The protocol does not implement its own bridge validator set.

30.3 16.3. Event Subscription (Ethereum to Cowboy)

- Cowboy actors MAY subscribe to event logs emitted by specific contracts on the Ethereum blockchain.
- Event subscriptions are handled via the bridge validator set (§16.2), which acts as a decentralized oracle monitoring the Ethereum chain for specified events.
- When a subscribed event is confirmed (i.e., finalized on Ethereum), the bridge infrastructure MUST enqueue a message to the subscribing Cowboy actor, delivering the event's topic and data as the message payload.

- The cost of this subscription service SHALL be paid by the actor in CBY, covering the gas fees incurred by the oracle validators on Ethereum.

30.4 16.4. Policy and Security

- All interoperability functions available to an actor, such as `bridge_asset` or `subscribe_event`, MUST be governed by the Entitlements system (§15).
- An actor's deployment manifest MUST declare the specific Ethereum contracts it is permitted to interact with and the types of assets it is allowed to bridge, enforcing the principle of least privilege.

31 17. Fee Model Specification

This section is authoritative; any conflicting values elsewhere are non-normative.

31.1 17.1. Overview

Cowboy uses a **dual-metered fee system**:

Meter	Unit	Purpose
Cycles	Compute units	CPU time, opcode execution, actor API calls
Cells	Data units (bytes)	Storage writes, calldata, bandwidth

Both meters use independent EIP-1559-style basefee adjustment. Fees are paid in CBY.

Three cost domains: 1. **On-chain execution** — Cycles consumed by transaction processing 2. **On-chain storage** — Cells consumed by state writes + ongoing state rent 3. **Off-chain services** — Direct CBY payments to Runners (LLM inference) and Providers (blob storage)

31.2 17.2. Transaction Intrinsic Costs

Every transaction pays a base cost before execution begins:

Transaction Type	Base Cycles	Base Cells	Notes
Transfer	10,000	0	EOA-to-EOA value transfer
Deploy	50,000	100/byte + <code>code_size</code>	Actor deployment
ActorMessage	10,000	<code>calldata_size</code>	Method invocation
LlmRequest	10,000	<code>prompt_size</code>	Off-chain inference request
TimerSchedule	10,000	64	Schedule future execution

31.3 17.3. Execution Costs (Cycles)

31.3.0.1 Opcode Costs Python opcode costs are implementation-defined and not protocol-specified. The runtime MUST ensure deterministic cycle consumption across all validators.

31.3.0.2 Actor API Costs

Operation	Base Cost	Variable Cost
<code>send_message()</code>	500 cycles	—
<code>storage_read()</code>	10 cycles	—
<code>storage_write()</code>	200 cycles	+ cells
<code>hash()</code>	100 cycles	+1 cycle/byte hashed
<code>verify_signature()</code>	10,000 cycles	—
<code>get_block_info()</code>	100 cycles	—
<code>emit_event()</code>	500 cycles	+5 cycles/byte
<code>set_timer()</code>	1,000 cycles	+ payload cells
<code>cancel_timer()</code>	500 cycles	—

31.3.0.3 Platform Token Costs (CIP-20)

Operation	Cycles	Cells
<code>token_transfer()</code>	1,000	64

Operation	Cycles	Cells
<code>token_transfer_from()</code>	1,500	96
<code>token_approve()</code>	500	32
<code>token_balance_of()</code>	100	0
<code>token_mint()</code>	1,000	64
<code>token_burn()</code>	500	64
<code>token_create()</code>	10,000	256 + name + symbol

Validation hooks add up to 50,000 cycles per transfer (capped).

31.4 17.4. On-Chain Storage Costs (Cells)

Operation	Cell Cost
State write	1 cell/byte written
State read	0.01 cells/byte (bandwidth metering)
Calldata	1 cell/byte of transaction data
Event emission	0.5 cells/byte of event data

31.5 17.5. State Rent

Accounts exceeding the grace threshold pay ongoing rent:

$$\text{rent_per_rent_epoch} = \max(0, \text{account_size} - \text{grace_threshold}) \times \text{rent_rate}$$

Parameters:

```

grace_threshold = 10,240 bytes (10 KB)
rent_rate       = 0.001 CBY/byte/year (governance-adjustable)
rent_epoch_length = 1 day
eviction_threshold = 10 rent-epochs unpaid rent

```

Grace period behavior: - Accounts ≤ 10 KB: No rent charged - Accounts > 10 KB: Rent charged on excess bytes only - Unpaid rent accumulates as debt against the account - Eviction

after 10 rent-epochs of accumulated debt (state archived to blob storage, recoverable upon debt repayment)

31.6 17.6. Off-Chain Storage (CIP-9)

Large data (images, datasets, AI inference traces, runner working state) lives in CBFS-backed Volumes, billed as per-byte per-epoch storage rent:

Cost Component	How Charged
StorageCommitment	~64 bytes on-chain → Cell cost + state rent
Volume storage rent	$\text{used_bytes} \times \text{erasure_overhead} \times \text{storage_rent_rate}$ per epoch, settled on-chain via Relay Node billing attestations
Runner read transfer	Per-byte transfer fee from Runner to serving Relay Node

Volume data itself is **not cell-metered**; CBFS Relay Nodes meter usage externally and settle rent on-chain via attestations. See CIP-9 for full specification of: - Volume lifecycle, CapTokens, and scoped access control - Relay Node staking, health, capacity, and shard placement - Erasure coding, autonomous repair, and challenge-based availability - Storage rent billing, grace periods, and eviction

31.7 17.7. Off-Chain Compute (Runner Marketplace)

LLM inference is **not gas-metered**. Runners operate in a competitive marketplace:

Aspect	Specification
Pricing	Runners post quotes (CBY per token, per model)
Selection	Users specify max_price in LlmRequest; matching via auction or direct selection
Settlement	On verified delivery: 89% to runner (split with delegators per CIP-13), 10% burned, 1% to Treasury

Aspect	Specification
Collateral	<code>effective_stake >= max(10,000 CBY, 1.5 × declared_max_job_value)</code> (self-bond + delegated)
Verification	Attestation + random re-execution challenges

The protocol does NOT specify LLM pricing—this is determined by market dynamics between users and runners.

31.8 17.8. Fee Adjustment (EIP-1559 Style)

Both Cycles and Cells use independent basefee adjustment:

$$\text{next_basefee} = \text{basefee} \times (1 + \delta \times (\text{usage} - \text{target}) / \text{target})$$

Cycle parameters: | Parameter | Value | | Target | 10,000,000 cycles/block |
 | Cap | 20,000,000 cycles/block | | δ (delta) | 0.125 (12.5% max change) | | α (smoothing) | 8 blocks |

Cell parameters: | Parameter | Value | | Target | 500,000 bytes/block | | Cap | 1,000,000 bytes/block | | δ (delta) | 0.125 (12.5% max change) | | α (smoothing) | 8 blocks |

Basefee burning: 100% of basefee revenue is burned, creating deflationary pressure proportional to network usage.

31.9 17.9. Reserved Capacity (Execution Lanes)

Block space is partitioned to guarantee execution for critical transaction types:

Lane	Cycle Budget	Percentage	Purpose
Timer	2,000,000	20%	Scheduled actor execution
Runner	2,500,000	25%	LLM result callbacks
System	500,000	5%	Governance, upgrades
User	5,000,000	50%	Regular transactions

Lane behavior: - Unused capacity in reserved lanes spills to User lane - User lane cannot borrow from reserved lanes - Timer lane has highest priority within its reserved capacity; execution is still subject to GBA bidding and per-block limits

31.10 17.10. Fee Estimation

Wallets and applications SHOULD estimate fees as:

```
def estimate_fee(tx):
    intrinsic_cycles = INTRINSIC_COSTS[tx.type]
    intrinsic_cells = len(tx.calldata)

    # Estimate execution cost (via simulation or heuristics)
    execution_cycles = simulate_execution(tx)
    execution_cells = estimate_storage_writes(tx)

    total_cycles = intrinsic_cycles + execution_cycles
    total_cells = intrinsic_cells + execution_cells

    # Apply current basefees
    cycle_fee = total_cycles * cycle_basefee
    cell_fee = total_cells * cell_basefee

    # Add priority tip
    priority_fee = (total_cycles * tip_per_cycle) + (total_cells * tip_per_cell)

    return cycle_fee + cell_fee + priority_fee
```

32 Appendix A. Transaction Encoding Test Vectors (Informative)

These vectors specify **canonical CBOR** encodings. Hex is lowercase, no 0x prefix.

Vector 1: Unsigned transfer (signature = null)

Fields:

- chain_id=1
- nonce=0

- to=0x11
- value=1
- cycles_limit=21000
- cells_limit=0
- max_fee_per_cycle=10
- max_fee_per_cell=2
- tip_per_cycle=1
- tip_per_cell=0
- access_list=null
- payload=0x (empty bytes)
- signature=null

CBOR hex:

[illegible]

The signing hash is `keccak256(CBOR(Tx_without_signature))` where the `signature` field is `null` (as above).

Vector 2: Signed transfer

Same fields as Vector 1, with:

- signature=[y_parity=0, r=0x01*32, s=0x02*32]

CBOR hex:

[illegible]

End of specification.